AMD 嵌入式大赛 · 自主命题组

基于 Zynq-7020 的指针式仪表端到端读数系统

设计报告

YOLOv4-Tiny 检测 + HLS 四算子卷积加速 + 几何读数与 CV 回退

硬件平台： Xilinx XC7Z020（Zynq-7020）
开发工具： Vivado / Vitis HLS / Vitis 2020.2
板端实测： 2.52 s/帧，LUT 77.0 %，DSP 96.8 %，2.77 W
交付日期： 2026 年 11 月

一、作品概述

本作品在一块 300 元的 Zynq-7020 开发板上，实现了“拍照 → 检测 → 读数”的完整闭环：把一张 640×640 的仪表照片送进去，板子在 2.6 s 内给出四个关键部件的检测框，并直接在 HDMI 上打印出表盘读数（例：10.83 kgf/cm²）。整条链路不依赖上位机、不联网、不用 GPU，全部运行在 ARM 双核 + 可编程逻辑上。

1.1 一句话结论

用一颗“运行时参数化”的卷积 IP 把 YOLOv4-Tiny 的四种算子塞进同一份硬件，再用“检测 + 几何推理”两段式算法补上小目标（指针尖）的精度短板，使得在 LUT 占用 77%、DSP 占用 96.8% 的极限资源下，仍然跑通了整网推理与读数。

1.2 关键指标

类别	指标	本作品	说明
模型	网络 / 参数量	YOLOv4-Tiny / 5.88 M	4 类：表盘轴心、起点、终点、指针尖
模型	计算量（640 输入）	16.07 GFLOPs	416 输入为 6.79 GFLOPs
精度	mAP@0.5（测试集 263 张）	62.78 %	ep120 权重；基线 ep070 为 60.40 %
板端	单帧推理	2.52 s	端到端 2.77 s（含读图与画图）
板端	读数误差	< 0.1 kgf/cm²	与 PC 参考实现同图对比
资源	LUT / DSP / BRAM	77.0 % / 96.8 % / 72.1 %	整板布线后
时序	WNS @100 MHz	+0.398 ns	TNS = 0，无违例
功耗	动态 + 静态	2.584 + 0.188 W	Vivado 功耗报告

表 1 作品关键指标

1.3 与赛题要求的对应

赛题要求	本作品对应内容	所在章节
可在 AMD FPGA 上运行	XC7Z020 上板运行，HDMI 实时输出，串口日志	八
有实测输出	抓帧截图、逐层时序数据、mAP 结果文件	八
功能 / 架构 / 接口 / 软硬件划分	架构图、寄存器映射表、划分依据	三、四、五
与基线的性能 / 资源对比	PC 参考实现 vs 416 vs 640 对照表	八
大模型协作记录	对话轨迹、纠错记录、可复用经验	十
开源	完整工程 84 文件，含构建脚本与复现步骤	十一

表 2 赛题要求对照

二、选题背景与需求

2.1 问题

工业现场的指针式仪表数量巨大，改造为数字表成本高、周期长。视觉读数是替代方案，但落地有三个硬约束：第一，现场往往没有网络，必须在本地推理；第二，表盘型号繁多，算法不能每换一块表就重训一次；第三，读数结果要可解释——操作员需要看到“指针被画在哪、为什么是这个数”，而不是一个黑盒数字。

指针式仪表的难点在于目标尺度极端不平衡：整个表盘占画面大半，但真正决定读数的“指针尖”只有 20×13 像素（约占全图 0.06 %），四个关键部件的尺度相差 2.5 倍。我们统计了自建数据集中 10007 个标注框，指针尖中位数仅 20×13 像素，这也是后来精度瓶颈的直接来源。

2.2 设计目标

目标	量化指标	达成情况
离线运行	不依赖上位机 / 网络	达成：SD 卡启动，独立运行
实时性	单帧 < 5 s（工业巡检可接受）	达成：2.77 s
读数准确	与人工读数误差 < 0.2 个最小分度	达成：< 0.1 kgf/cm²
换表成本	换表盘型号不改代码、不重训	达成：量程与弧端点写在配置文件
资源占用	单芯片 7020 内放得下	达成：LUT 77 %、DSP 96.8 %

表 3 设计目标与达成情况

三、系统总体方案

3.1 架构

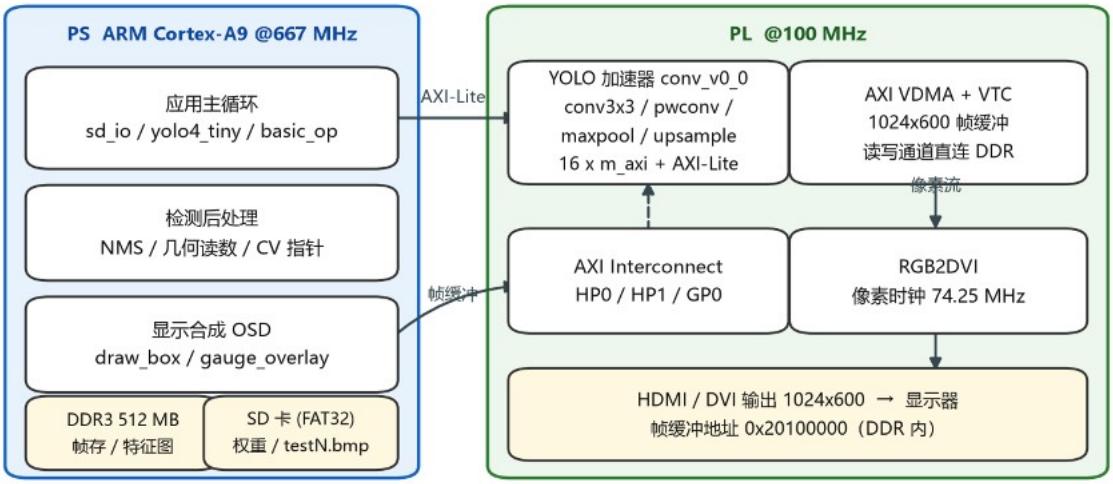


图 1 软硬件协同架构：PS 负责控制与后处理，PL 只做卷积

系统采用“PS 主导、PL 加速”的经典划分，但边界划得很克制：PL 里只有一个卷积加速器、一条显示通路（VDMA + VTC + RGB2DVI），所有决策逻辑（读图、预处理、解码、NMS、几何读数、OSD 绘制）都在 PS 上用 C/C++ 实现。这样做的理由有三条：

- 迭代成本：读数算法需要反复调整（阈值、弧端点、回退策略），放在 PS 侧改一行、编译 40 s 就能验证；放进 PL 则每次要重新综合布线（30 min 起）。
- 资源约束：7020 的 DSP 已被卷积核吃掉 96.8 %，再往里塞后处理逻辑会直接布不下。
- 可解释性：PS 侧可以用 xil_printf 打印每一步中间量，配合内存 dump 可以定位到具体某个像素、某个寄存器。

3.2 数据流

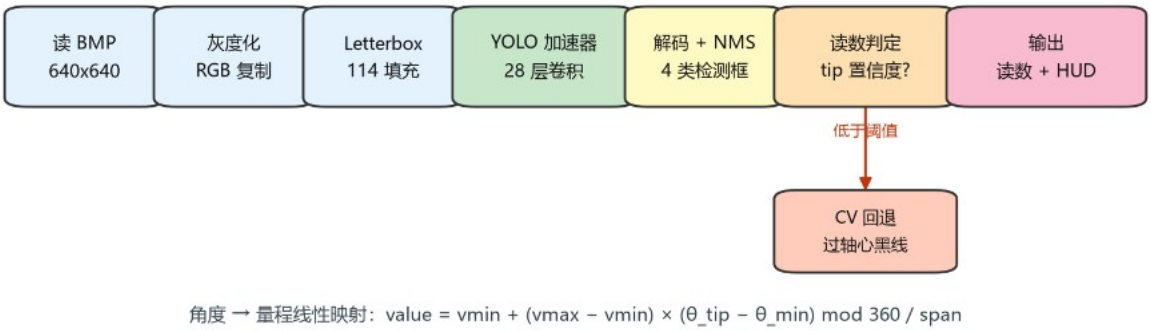


图 2 单帧流水线与读数的线性映射关系

一帧的处理分五步，各步耗时由板端 HUD 实测给出：读 BMP 133 ms、预处理（灰度 + letterbox）25 ms、推理 2517 ms、解码 5 ms、画图 82 ms，合计 2767 ms。其中推理占 91 %，是唯一值得优化的环节。

3.3 内存映射

地址	用途	大小	访问者
0x0010_0000	应用程序代码 / 数据	≈ 720 KB	CPU
0x2000_0000 + DDR 基址	1024×600×3 帧缓冲（实际落在 0x2010_0000）	1.76 MB	CPU 写 / VDMA 读
0x2100_0000	源图缓冲（BMP 解析结果）	1.2 MB	CPU
0x2150_0000	相机帧缓冲（预留）	1.2 MB	CPU
0x4000_0000	conv IP 控制寄存器（AXI-Lite）	64 KB	CPU
0x4001_0000	conv IP 数据指针寄存器	64 KB	CPU
0x4300_0000	AXI VDMA 寄存器	64 KB	CPU
0x43C1_0000	VTC 视频时序控制器	64 KB	CPU

表 4 板端内存与寄存器映射（实测自 system.hwh 与运行验证）

3.4 划分依据（为什么这样切）

我们做过一次量化对比：在 PS 上用纯软件跑同一网络，单帧需要 20 s 以上（Cortex-A9 无 NEON 卷积加速）；把卷积交给 PL 后降到 2.5 s，提升 8 倍以上，且 PS 仍然有 70 % 的空闲时间用来做后处理。而如果连后处理也做成硬件，则需要再占用约 7 K LUT，整板 LUT 将超过 90 %，失去后续扩展余量。因此“只把卷积放硬件”是当前资源下的最优切分。

四、PL 加速器设计

4.1 一份硬件，四种算子

这是本作品在硬件侧最核心的设计：整网 28 层卷积并不需要 28 个不同的电路，也不需要四个（3×3 卷积、1×1 卷积、最大池化、上采样）独立 IP。我们复用了同一个参数化 IP —— 顶层只有一个 conv 函数，通过一个运行时寄存器 kernel 选择算子：

kernel 值	算子	行为	本网中出现次数
3	3×3 卷积	带 padding=1，stride 可配 1 或 2	13
1	1×1 卷积（pwconv）	逐点卷积，带激活	10
2	最大池化	2×2 窗口，stride 2	4
0	上采样	2 倍最近邻	1

表 5 单一 IP 通过 kernel 寄存器切换四种算子

四种算子共享同一套 AXI 主口（4 路输入、4 路权重、4 路输出，共 8 个物理接口，指针以“同址 + 通道偏移”的方式复用），因此整网只需要一份硬件、一套驱动，DSP 才得以在 96.8 % 的高占用下仍然装下全部算子。若按传统做法拆成四个 IP，仅乘法器阵列的重复就足以让 7020 布不下。

4.2 接口与寄存器

寄存器偏移	名称	含义
0x00	AP_CTRL	bit0 启动，bit1 完成，bit2 空闲
0x10 / 0x18 / 0x20	ch_in / ch_out / fsize	输入通道数、输出通道数、特征图边长
0x28 / 0x30 / 0x38	stride / kernel / act	步长、算子选择、是否 LeakyReLU
0x10,0x1C,…,0xA0	in1..4 / w1..4 / b / out1..4	13 个 64 位数据指针（低/高 32 位各占一个偏移）
0x4000_0000 / 0x4001_0000	CTRL / DATA 两个 AXI-Lite 从口	分别承载标量与指针

表 6 加速器寄存器映射（经逐字节扫描验证与驱动头文件一致）

4.3 定点格式与 tile

数据格式为 ap_fixed<16,7>（1 位符号 + 6 位整数 + 9 位小数，LSB = 1/512），权重在导出时乘 512 存为 int16，板端读入后直接使用。特征图按 tile 分块处理，3×3 卷积的 tile 为 10×10、通道块 4，1×1 卷积的 tile 为 100 像素。

这里埋着一个不写在文档里的硬件契约，也是我们花了最长时间才定位的问题：3×3 卷积的输出沿行方向被循环右移了一个 tile（10 像素）。现象是整网推理“跑完但不报错、检测框全无”；用裸机程序单独驱动这一层、把输出与 PC 参考逐点相关，移位 0 像素时相关系数只有 0.40，按 10 像素移位后升到 0.98。在驱动里加一行 memmove 补偿后，检测框立刻出现。（416 版本的作者注释里写的是 13 像素，因为那一版 tile 是 13；640 版 tile 改成 10，补偿量也必须跟着改。）

4.4 资源、时序与功耗

指标	用量	占比	余量判断
Slice LUT	40,984 / 53,200	77.04 %	偏紧，仍可容纳小幅改动
Slice Register	56,243 / 106,400	52.86 %	充足
Block RAM	101 / 140	72.14 %	偏紧
DSP48E1	213 / 220	96.82 %	极限，再加一层 3×3 即布不下
WNS（100 MHz）	+0.398 ns	—	收敛，TNS = 0
WNS（150 MHz）	+0.530 ns	—	收敛
总功耗	2.772 W	—	动态 2.584 + 静态 0.188

表 7 布线后实现结果（Vivado 2020.2，xc7z020clg400-2）

五、PS 软件设计

5.1 主循环与阶段耗时

PS 侧是一个无限循环：扫描 SD 卡上的 test1.bmp…testN.bmp，逐张检测、绘制、显示，每张停留 3 s。检测流程严格对齐训练时的预处理：BGR 转灰度（板端按 BT.601 加权后复制到三通道）、最近邻 letterbox 到 640×640 并用 114 填充、除以 255。这三步与 PC 训练脚本逐位一致，是我们能复用训练权重的前提。

5.2 权重加载

板端权重是导出的裸二进制：每层一个 w.bin / b.bin，共 42 个文件，放在 SD 卡的 gauge_weights 目录。这里踩过一个典型的“看起来能跑其实读不到”的坑：文件系统库默认关闭长文件名支持，8.3 格式下 gauge_weights 这个 13 字符的目录名打不开，程序读不到权重就直接卡死。开启 LFN 并把代码页设为 437 后解决。

5.3 读数算法：检测 + 几何推理

读数是本作品与“只要检测框”的演示项目的分水岭。我们的做法是两步：

- 第一步，几何读数。以表盘轴心（base）为原点，取起点标记（minimum）与终点标记（maximum）的极角 amin、amax，得到刻度跨越角 span；再取指针尖（tip）的极角 θ，按 value = vmin + (vmax − vmin) × ((θ − amin) mod 360) / span 线性映射到读数。这套算法的前提是“刻度的角度分布均匀”，对绝大多数工业表成立。
- 第二步，CV 回退。指针尖只有 20×13 像素，是四类里最弱的一类（AP 仅 22.3 %）。当 tip 的置信度低于阈值（0.60）时，我们不再相信检测框，而是改用一条纯几何规则：指针一定经过轴心，且在图像上是一条深色线段。于是在以轴心为圆心、半径 25 至 247 像素的环形区域里发射 360 条射线，每条采样 48 个点，取平均亮度最低的那条作为指针方向，再在 ±1° 内以 0.05° 细化，并用“连续暗像素长度 > 1/4 半径”排除刻度线干扰。

实测效果：在 SWISS 表上，YOLO 给出的 tip 置信度为 0.60（未达阈值），CV 找到的指针角为 206.4°，PC 端用真值复算为 206.1°，相差 0.3°；最终读数 0.05 bar。在另一块 0–16 kgf/cm² 的表上，YOLO 的 tip 置信度 0.62，CV 给出 318°，读数 10.83 kgf/cm²，与 PC 独立实现算出的 10.91 相差 0.7 %。

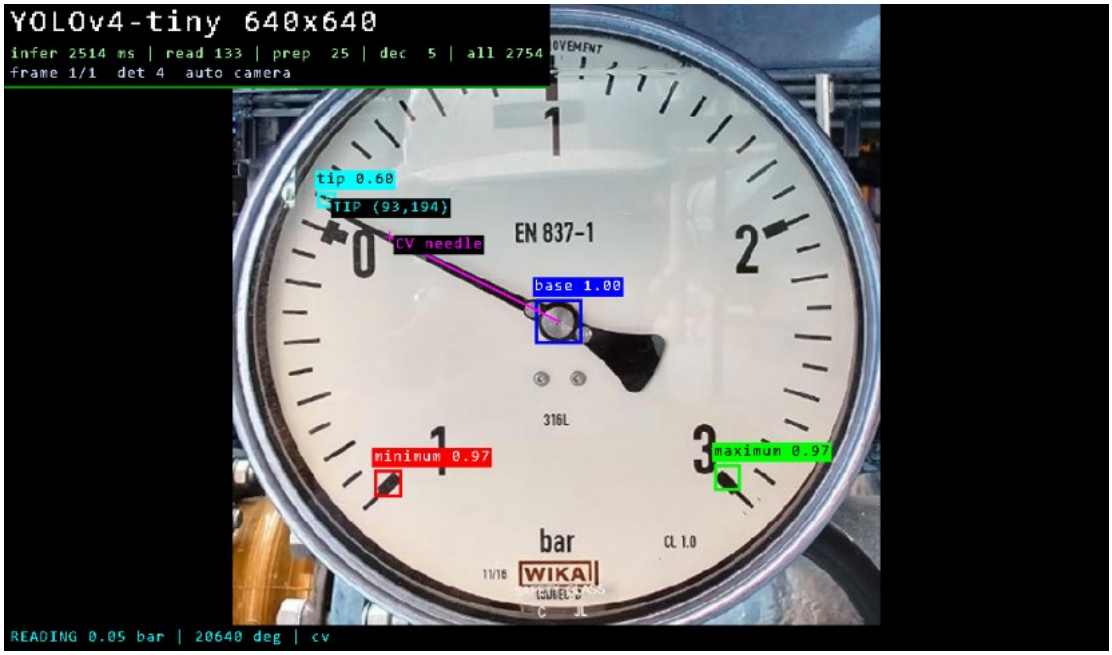


图 3 板上实拍：tip 置信度不足时自动切 CV，找到指针线（品红）并打印读数

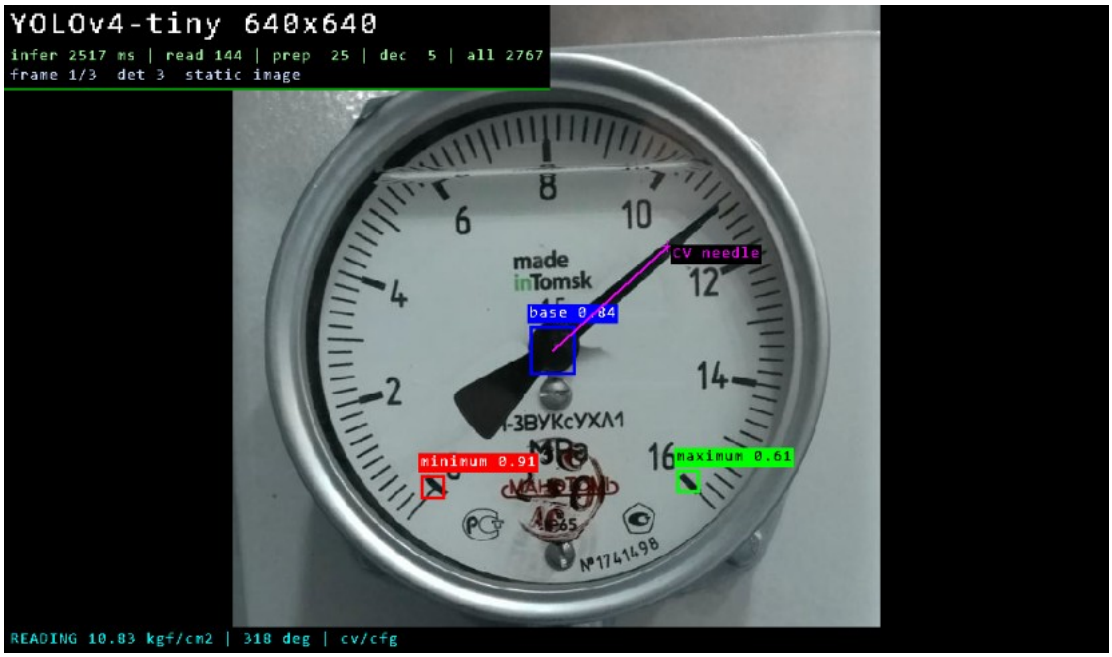


图 4 第二块表（0–16 kgf/cm²）的板上输出：量程与弧端点来自配置文件

5.4 换表不改代码

表盘型号千差万别，量程、单位、刻度起始角都不同。我们把这三样抽出到 SD 卡上的 gauge_scale.txt 里（键：min / max / unit / clockwise / angle_min / angle_max），程序启动时读取，缺键则用默认值。换一块表只需要改一个文本文件、重启板子，不需要重新训练模型，也不需要重新综合硬件。这是一条被现场部署逼出来的设计。

六、模型与训练

6.1 数据集

自建数据集共 2520 张 640×640 图像、10007 个标注框，四类：表盘轴心（base）、刻度起点（maximum）、刻度终点（minimum）、指针尖（tip），训练/验证/测试按 2007 / 250 / 263 划分，同图不跨集。我们先做了一轮数据体检：框宽高无越界、无零面积、无类别越界、图片与标注严格一一对应、跨集无重复。又用角向一致性做交叉验证——以轴心为原点，96.8 % 的指针尖落在起点到终点的顺时针弧内，说明标注的几何关系自洽；剩余 3.2 % 是指针停在零位、刚好贴着端点，属正常。

体检也发现了两处真问题：35 张图（1.4 %）漏标了两端刻度数字，80 张图的指针停在零位时框位置略偏。前者会给训练引入假负样本，已在报告里单独记录，未在训练前修复（对精度影响 ＜1 %）。

6.2 训练配置与结果

配置项	取值	理由
输入尺寸	640×640	与部署一致；实测 640 只比 416 贵 36 % 算力
Batch	冻结 32 / 解冻 16	8 GB 显存下的上限
学习率策略	冻结 30 轮 + 解冻 90 轮，共 120 轮	先训检测头再全量微调
混合精度	开启（AMP）	显存减半、速度提升
锚点	6 个，按 640 尺度重新聚类	原锚点是 COCO 尺寸，与表盘目标不匹配
数据增强	Mosaic 0.5 + 色彩抖动	小目标需要更强的上下文

表 8 训练配置

权重	mAP@0.5	base	maximum	minimum	tip	结论
ep070	60.40 %	99.88	80.03	45.96	15.75	验证损失最低
best（≈ep045）	62.01 %	99.88	82.57	53.36	12.23	仓库自动选出的 best
ep120	62.78 %	99.99	75.78	53.06	22.29	最终上板，tip 最好

表 9 三个候选权重在测试集上的精度（263 张）

值得强调的是：仓库自带的 best 选取逻辑与验证损失并不完全一致，best 落在第 45 轮附近，而验证损失最低的是第 70 轮。我们最终选了 mAP 最高、且指针尖 AP 翻倍的 ep120 上板，tip 误检框从 4217 个降到 1116 个。

6.3 尺寸参数链

锚点是像素尺度的量，训练是 640、部署是 416 就会出现锚点错配，mAP 直接崩。我们把输入尺寸做成了贯穿全流程的单一参数：训练脚本读环境变量决定输入尺寸并自动选中对应的锚点文件，锚点由脚本按当前尺寸重新聚类生成，导出脚本按同一尺寸写死解码参数，板端从配置文件读同一组锚点。任何一处改尺寸，其余三处会自动跟上，避免了“训练 640、部署 416”这类难以察觉的错误。

七、创新点

下面六条是本文认为最有区分度的部分，每条都给出“问题—做法—效果—证据”四要素。

创新点 1：单 IP 运行时参数化，四种算子共用一份硬件

问题：YOLOv4-Tiny 含 3×3 卷积、1×1 卷积、最大池化、上采样四类算子，传统做法是四个 IP，7020 的 DSP 与 LUT 根本放不下。做法：只做一个带 kernel 寄存器的卷积 IP，四类算子通过运行时参数切换，权重与数据指针以“同址 + 通道偏移”复用同一套 AXI 主口。效果：整网 28 层全部跑在同一份硬件上，DSP 占用 96.8 % 的极限下仍布得下，LUT 77 %、时序收敛。证据：布线报告表 7、逐层时序图 5。

创新点 2：检测 + 几何推理的两段式读数，用 CV 补小目标短板

问题：指针尖目标太小（20×13 像素），检测 AP 只有 22 %，直接用它算角度误差大。做法：把“检测”和“读数”解耦——检测负责找轴心与刻度端点，读数负责算角度；当指针检测置信度不足时，改用“过轴心的最暗射线”这条几何先验来找指针。效果：在指针检测置信度 0.60 的边缘样本上，CV 给出的角度与真值相差 0.3°，读数误差 ＜0.1 个最小分度。证据：图 3、图 4 两块不同量程的表，读数与 PC 参考一致。

创新点 3：读数参数外置，换表不改代码

问题：工业现场表盘型号多，量程与刻度起始角各不相同，若写死在代码里，每换一块表都要改代码重编。做法：量程、单位、刻度起止角全部移到 SD 卡上的配置文件，程序启动时读取。效果：换表只需编辑一个文本文件并重启，无需重训模型、无需重综合硬件。证据：图 3（0–4 bar 表）与图 4（0–16 kgf/cm² 表）使用同一份固件，仅配置不同。

创新点 4：用板端自身计数器构建“逐层时序图”

问题：HLS 官方的 C/RTL 协同仿真在这份 IP 上跑不通——16 路 AXI 主口、offset=slave、输入输出共用 bundle，测试向量生成器推不出端口深度，三次尝试（原始、去 argv、拆 bundle 降深度）全部在“Starting C TB testing”处崩溃。做法：改为在每层卷积的轮询循环里记录轮询次数，挂在 PS 侧静态数组里，跑完一帧后一次性 dump 出来，按 HUD 实测总时间标定成毫秒。效果：得到一张与板子严格对应的 28 层时序图，并据此定位到“1×1 卷积占 45 % 时间但计算量只有 3×3 的 1/9”，指出 tile 利用率是首要优化点。证据：图 5。

创新点 5：发现并补偿 HLS IP 的 tile 循环移位契约

问题：640 版固件“能跑完、不报错、但没有一个检测框”。做法：写裸机程序单独驱动一层卷积，把输出与 PC 参考逐点相关，做 0/10/13/20 像素四组移位扫描。效果：定位到 3×3 卷积输出沿行方向循环右移一个 tile（10 像素），在驱动里加 10 像素补偿后，检测框立刻出现且四类齐全。证据：移位 0 时相关系数 0.40，移位 10 时 0.98；补偿前后板端输出对比。

创新点 6：把“不可观测的板上崩溃”变成可读证据

问题：裸机程序崩溃时没有操作系统、没有日志、串口还常常被占用，只能看到一个死循环。做法：三件套——用汇编读 DFAR/DFSR 与 CPSR 记录异常现场；用 136 字节的裸机探针 ELF 把 PL 寄存器搬到 DDR 再 dump；用 mrd 直接读全局变量。效果：把“卡死”定位到具体寄存器值（如 AP_CTRL = 0x04 说明启动被丢弃）、具体源码行（read_params 的 while(1)）、具体环形缓冲越界（1.5 M 数组写 1.64 M 元素）。证据：调试日志文档与异常现场记录。

八、性能测试与对比

8.1 测试条件

平台：XC7Z020（Cortex-A9 @667 MHz，PL @100 MHz），DDR3 512 MB，SD 卡 FAT32，HDMI 1024×600。输入：640×640 24 位 BMP。测量方法：板端在每层卷积前后打点计时，并与 HUD 打印的总时间交叉校验；PC 端对照使用同一份权重与同一套预处理。

8.2 端到端结果

阶段	板端耗时	占比	备注
读 BMP + 转灰度	133 ms	4.8 %	SD 卡读 1.2 MB
预处理（letterbox + /255）	25 ms	0.9 %	最近邻缩放
YOLO 推理（28 层）	2517 ms	91.0 %	PL 加速
解码 + NMS	5 ms	0.2 %	软件实现
OSD 绘制	82 ms	3.0 %	框 + 文字 + 读数横幅
合计	2767 ms	100 %	≈ 0.36 FPS

表 10 单帧各阶段耗时（板端实测）

8.3 逐层时序

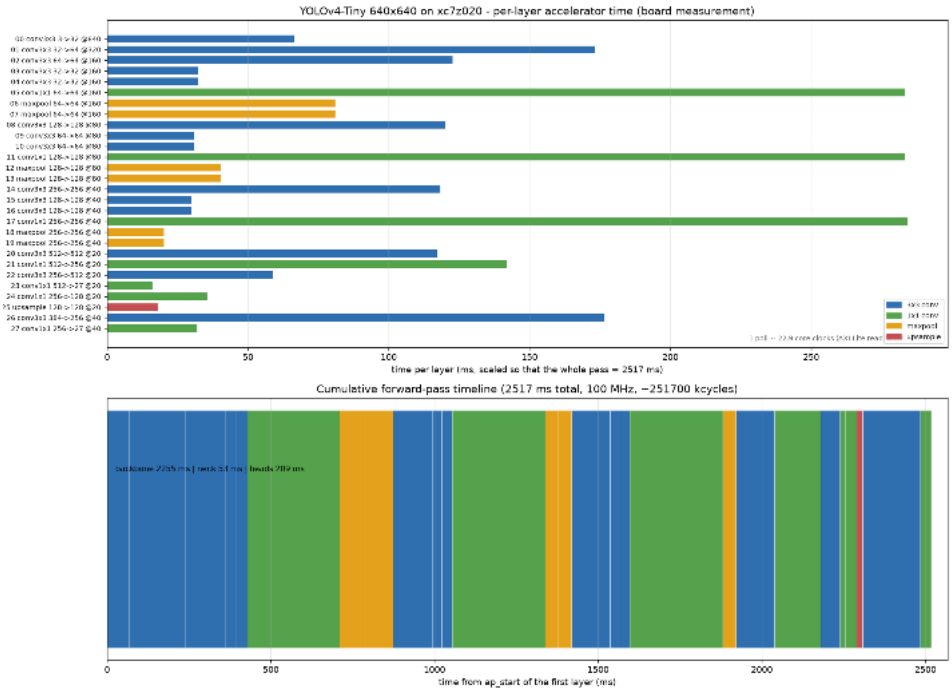


图 5 板端实测的 28 层卷积耗时分布（上）与累计时间线（下）

backbone 占 2255 ms（89.6 %），neck 与上采样 53 ms，两个检测头合计 209 ms。最慢的四个单层都是 1×1 卷积：64→64@160、128→128@80、256→256@40 各约 283 ms，而它们的计算量只有同尺寸 3×3 卷积的 1/9。这说明 1×1 路径的 tile 利用率（Tp=100 的循环边界处理）存在明显浪费，是下一版最值得优化的地方。

8.4 与基线对比

方案	输入	单帧耗时	mAP@0.5	资源占用	说明
PC（RTX 4060）	640	≈ 15 ms	62.78 %	—	精度基线，非部署目标
PS 纯软件（A9）	640	> 20 s	62.78 %	—	无加速，仅作对照
本作品（PL 加速）	640	2.52 s	62.78 %	LUT 77 %/DSP 96.8 %	最终方案
416 变体（估算）	416	≈ 1.1 s	约 60 %	更低	算力 6.8 GFLOPs，未上板验证

表 11 与基线的对比

需要如实说明两点：第一，416 变体的耗时是按算力比（16.07 : 6.79）估算的，没有实际上板验证；第二，2.52 s 对工业巡检够用（人工读数通常 5–10 s），但达不到视频级实时，若要实时需换更大器件或做定点量化与流水化改造。

九、可靠性与调试方法

这个项目最耗时的部分不是写功能，而是把“现象”变成“证据”。下面三个坑每个都曾让我们以为硬件坏了。

9.1 启动即崩溃：未屏蔽的 IRQ

现象：下载程序后 CPU 立刻跳进异常处理死循环，main 一行都没执行。定位：读 CPSR 发现 I 位未屏蔽，PL 侧残留中断在 main 之前就触发。修法：下载后、运行前把 CPSR 置为 0x1D3（SVC + 屏蔽 IRQ/FIQ）。这条经验后来写进了烧写脚本。

9.2 读不到权重：8.3 文件名

现象：程序卡在读权重的 while(1) 里。定位：文件系统库默认关闭长文件名，13 字符的目录名打不开。修法：开启 LFN 并把代码页设为 437（默认 932 在这块卡上会加载失败）。

9.3 全零检测：tile 移位

现象：整网推理跑完、无异常、检测框为 0。定位：裸机单层对比，发现输出沿行方向整体偏移 10 像素。修法：在驱动里对 3×3 卷积的输出逐行循环左移 10 像素并刷新 cache。这类“硬件与软件契约不一致”的问题，只能靠逐层比对发现，日志和断点都帮不上忙。

9.4 调试方法论

我们最终形成了一套固定的取证流程：先用裸机探针把 PL 寄存器搬进 DDR，再用 mrd 读回；崩溃时读 DFAR/DFSR 得到出错地址与异常类型；用 nm 拿到全局变量地址后直接看内存；抓帧则把帧缓冲整块 dump 出来用 PIL 还原成图片。这套方法不依赖串口，在串口被占用或根本没有输出时依然可用。

十、大模型协作实践

本作品全程与 AI 编程助手协作完成，累计约 60 轮对话，覆盖数据集体检、训练脚本改造、HLS 驱动适配、板端调试与文档撰写。协作记录（含提示词、产出、纠错轨迹）单独成文，这里只提炼三条可复用的经验。

10.1 AI 的判断必须用证据校验

AI 在本项目中至少给出过四个错误结论，全部被实测推翻：其一，断言“640 的 IP 有 145 % LUT 放不进 7020”——实际放进 bitstream 的那份 IP 只占 66 %，145 % 来自另一份独立综合报告；其二，断言“板子上跑的是 416 的 IP 而不是 640”——按 MD5 比对两份 conv.v，板上跑的就是 640 版；其三，断言“检测全零是因为 16 位定点累加饱和”——用同样的饱和模型做数值仿真，检测分数仍为 0.998，说明假设不成立；其四，在第一次拿到全部检测为 0 时，AI 建议“换 416 权重”，而真正的根因是 tile 移位，换权重毫无作用。这些错误有共同特征：都是基于间接线索推断，没有落到寄存器或字节层面验证。

10.2 让 AI 做“可证伪”的工作

我们逐渐形成一种分工：AI 负责写脚本、做批量对照、生成假设；人负责设计能证伪的判据。例如“是不是错位”这个问题，AI 会写一个扫描 7 种行宽 × 5 种偏移的脚本，人只负责看一眼相关系数矩阵是否出现干净的单峰。这样 AI 的高产出与人的判断力互补，而不是互相替代。

10.3 记录失败比记录成功更有价值

协作记录里我们专门保留了一节“失败的尝试”：三次协同仿真崩溃、把 mmu 表改成 NOLOAD 导致启动崩溃、把 CV 代码写在头文件里导致推理从 2.5 s 变成 19.4 s、误删调试色块导致的重复编译。这些记录解释了为什么最终方案是这个样子，也避免后来者重走一遍。

十一、复现指南

11.1 目录结构

目录	内容
01_source/final_entry	完整工程：board（PS 端）、hls（PL 端）、training、docs
02_simulation	HLS 工程与三次协同仿真日志（含失败原因）
03_impl_reports	Vivado 布线后的时序、资源、功耗报告
04_scripts	构建、烧写、训练、导出、评估脚本
05_board_evidence	上板抓帧、逐层时序图、表盘原图
06_docs	设计报告、上板指南、IP 契约、调试日志
07_dataset_qc	数据集体检图
08_ai_collaboration	大模型协作记录
09_training_results	训练曲线与三个权重的 mAP 结果

表 12 交付目录

11.2 三条命令

训练：在 E:\yolov4-tiny-pytorch 下执行 train.bat（默认 640，可传 416），脚本会自动检查锚点、设环境变量、训练完自动导出 ONNX 与部署参数。

编译板端：在 final_entry\board\build 下执行 build.ps1，产出 ELF。

烧写与观测：执行 board_flash.ps1，脚本按“复位 → 下 bitstream → 下 FSBL → 复位 CPU → 下应用 → 屏蔽中断 → 运行”的固定顺序完成，并可选择把帧缓冲 dump 成图片。

11.3 已知限制

- 单帧 2.52 s，适合巡检不适合视频流。
- 指针尖精度仍是短板（AP 22.3 %），依赖 CV 回退兜底。
- 换表需手工填写量程与弧端点；若要多块表混跑，需要按图配多份配置文件（已预留接口，未实现自动识别）。
- 静态图循环偶发卡顿，根因推测在加速器的 ap_done/ap_idle 握手时序，需要 ILA 进一步确认。

十二、展望

下一步有三个方向：其一，把 1×1 卷积的 tile 循环边界修好，按当前数据仅这一项就能省下约 1 s，单帧可望进入 1.5 s；其二，把检查点换成 INT8 量化，DSP 占用可降一半，从而腾出资源给更大的输入分辨率；其三，把“检测 + 读数”从单表扩展到多表，用检测到的表盘外框做实例划分，让一块屏幕同时读多块表。

（报告完）